CS 486/686
From Prompting to Fine-Tuning and LoRA

Yuntian Deng

Lecture 22

How to adapt and augment a language model

Search › Uncertainty › Decisions › **Learning**

Learning goals

- Choose among **prompting, RAG, tools, and fine-tuning**.
- Explain how **RLHF and DPO** learn from preferences.
- Derive the shapes and parameter savings of **LoRA**.
- Explain how a large compiler can generate an adapter for a small model.

From L21 to L22

L21 showed how to run one fixed Qwen model.

instructions	**knowledge**	**actions**	**behavior**
change the prompt	retrieve evidence	call a tool	fine-tune weights

Diagnose what is missing, then choose the smallest useful lever.

Suppose we are building a CS486 course helper

STUDENT When and where is the final exam?

HELPER Saturday, August 8, 7:30–10:00 PM in PAC 5. *Source: course schedule.*

The answer must be current, correct, and grounded—not merely fluent.

How should the helper answer?

1. Find the relevant schedule entry.
2. Place that evidence beside the question.
3. Generate the answer and cite the source.

This question needs external knowledge at inference time: RAG.

The adaptation ladder

1	2	3	4
prompt	**RAG**	**tools**	**fine-tuning**
change instructions	add knowledge	add actions	change weights: full or LoRA

LoRA is a parameter-efficient way to fine-tune—not a separate adaptation goal.

Prompting changes context, not weights

BEFORE		AFTER
Answer the student.	→	Answer in two sentences. Quote the date and cite the course schedule.

The prompt controls how to answer; it still does not contain the date.

Prompt templates package behavior

ROLE	You are a concise CS486 course assistant.
EVIDENCE	Use only supplied course context; otherwise say “I don't know.”
FORMAT	Answer briefly and cite the source.

A reusable template standardizes behavior across questions.

A better prompt cannot invent a missing fact

When and where is the final exam?

no course schedule in context

retrieve the schedule first

ADAPTATION 1

Add current knowledge

Retrieval-augmented generation (RAG)

RAG puts evidence beside the question

QUESTION	When and where is the final exam?
RETRIEVED EVIDENCE	Final exam: Sat Aug 8, 7:30–10:00 PM, PAC 5.
GROUNDED ANSWER	Saturday, August 8 in PAC 5. *[course schedule]*

RAG changes the context at inference time—not the weights.

Before questions arrive: store document embeddings

Compute once; store the vector with the original text and its source.

When a question arrives: retrieve, then answer

“Nearest” is the embedding geometry from L18.

RAG on real course facts LIVE

Try the Assignment 3 deadline or final-exam question; inspect the two nearest chunks.

Interactive on the live slides: choose “When is Assignment 3 due?” or “When and where is the final exam?” The demo embeds the question, ranks real course chunks, and forms a grounded prompt from the best evidence.

One query benefits from two kinds of match

“Is the exam in
PAC 5
?”

keyword match

finds the exact room code “PAC 5”

embedding match

connects “exam” with “final examination”

reranker

chooses the strongest evidence

Debug RAG in two checkpoints

1. **Did retrieval find the right source?**
   If not, fix documents, embeddings, or ranking.

2. **Is every answer claim supported?**
   If not, fix the prompt or generation.

RAG reduces unsupported answers; it does not guarantee truth.

ADAPTATION 2

Add actions

Some questions require computation—not more text.

Language models should not guess exact arithmetic

My scores are 82, 74, and 91 with weights 20%, 30%, and 50%. What is my weighted grade?

model alone

may produce plausible but wrong arithmetic

calculator

$$0.2(82) + 0.3(74) + 0.5(91) = 84.1$$

Let the model choose the operation; let a deterministic tool execute it.

A tool call becomes new evidence

1 · propose	`calculator("0.2*82 + 0.3*74 + 0.5*91")`
2 · execute	The calculator runs outside the language model.
3 · observe	`84.1` returns to the model.
4 · answer	Your weighted grade is 84.1%.

Tool use, step by step LIVE

The model calls a calculator instead of guessing the arithmetic.

Interactive on the live slides: step through the weighted-grade tool loop. The model emits a calculator call, the calculator returns 84.1, and the model answers from that observation.

Agents repeat the same loop

plan → act → observe → stop or continue

For consequential actions, permissions and human confirmation belong around the loop.

ADAPTATION 3

Change behavior

When the same behavior must hold
across many inputs, change the weights.

A prompt can be brittle across phrasings

“When is A3 due?”	“Deadline for the third assignment?”	“How long do I have for Assignment 3?”

Always answer briefly, cite the evidence, and decline if evidence is missing.

Fine-tuning teaches a durable response pattern through examples.

Supervised fine-tuning learns from ideal responses

INSTRUCTION	When is Assignment 3 due?
IDEAL RESPONSE	Tuesday, August 4 at 11:59 PM. *[course schedule]*

INSTRUCTION	Which textbook is required?
IDEAL RESPONSE	I do not have supporting evidence in the supplied context.

Training still minimizes next-token loss—on curated behavior examples.

Train on examples; evaluate on new phrasings

curate
instruction → ideal response

→

train
update model weights

→

evaluate
held-out phrasings

warning: success on memorized wording does not imply robust behavior.

Preference data compares two answers

When and where is the final exam?

PREFERRED	Aug 8, 7:30–10:00 PM in PAC 5. *[schedule]*
REJECTED	It is probably during exam week in August.

Preferences say which answer is better without writing one perfect target token by token.

RLHF learns a separate reward model

preferences
chosen > rejected

→

reward model
generation → scalar score

→

policy update
increase expected reward

Like a learned utility signal from our decision-making unit—but not literally a state-value critic.

DPO optimizes the preference directly

chosen answer
raise its relative log-probability

rejected answer
lower its relative log-probability

direct policy loss
no standalone reward model; no PPO loop

The policy/reference log-ratio acts as an implicit reward score.

Rafailov et al., *Direct Preference Optimization*, NeurIPS 2023.

PARAMETER-EFFICIENT FINE-TUNING

LoRA: fine-tune fewer parameters

Keep the base weights frozen;
learn a compact update.

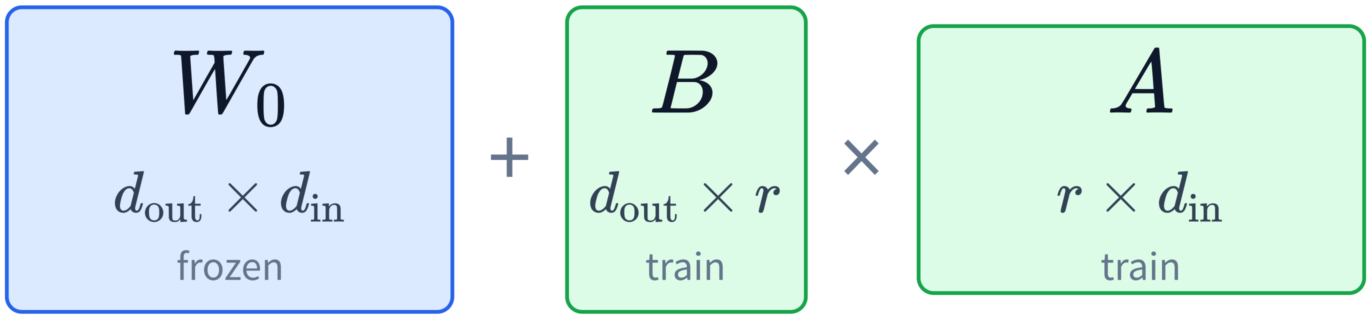
Full fine-tuning stores more than the weights

weights	gradients	AdamW state	AdamW state
W	∇W	first moment m	second moment v

Roughly four parameter-sized tensors before activations.

Mixed-precision training may also keep an FP32 master-weight copy.

LoRA factorizes a weight update



$$\Delta W = BA, \qquad W' = W_0 + \Delta W$$

d_{in}: input width d_{out}: output width r: adapter rank

For a square $d \times d$ matrix, choose $r \ll d$.

The base stays frozen; only A, B are learned

$$h = W_0 x + \frac{\alpha}{r} B A x$$

full square matrix

$$d^2$$

trainable parameters

LoRA matrices

$$dr + rd = 2dr$$

trainable parameters

Gradients and AdamW states are needed for A, B, not W_0.

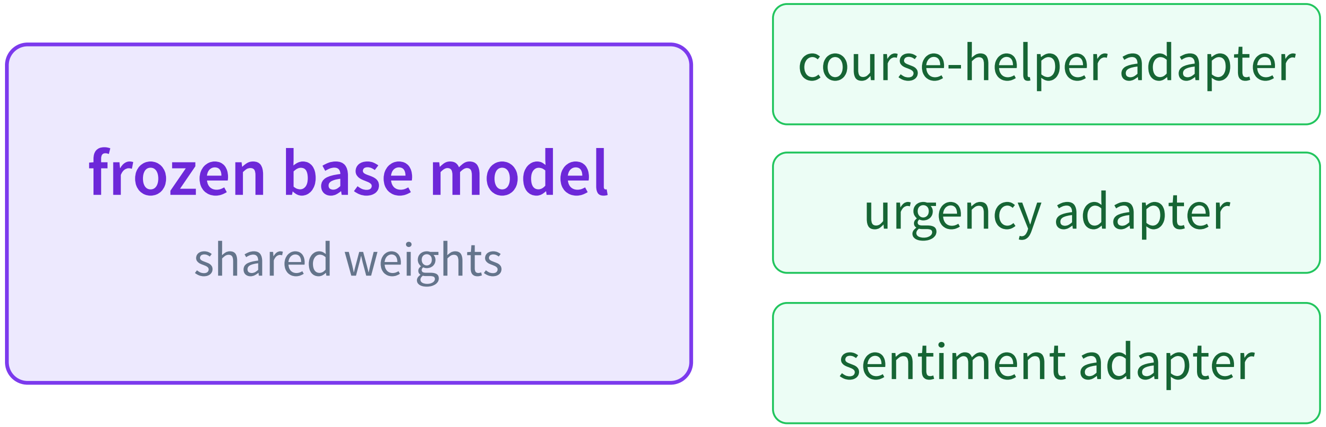
α/r controls the update scale.

LoRA: a low-rank update LIVE

Raise r from 1 to 24: expressivity grows, but parameter savings eventually disappear.

Interactive on the live slides: compare frozen W_0, $\Delta W = BA$, and W' as heatmaps while changing r from 1 to 24. The readout reports $2dr$ versus d^2, including ranks that are no longer parameter-efficient.

One frozen base can host many adapters



Save and swap small task-specific A, B matrices instead of duplicating the base.

Example: fine-tune an urgency classifier with LoRA

Collect examples of the behavior we want the adapter to learn.

"Thesis defense moved to 3 PM; need your signature today."	immediate
"The submission server is down right now."	immediate
"Please review this whenever you have time next week."	wait
"Newsletter draft for next month."	wait

The labels define the task; LoRA learns the mapping while W_0 stays frozen.

Then train and save the LoRA adapter

1 · define	2 · collect	3 · attach	4 · train	5 · evaluate
labels: `immediate` / `wait`	message → label examples	LoRA matrices A, B	update only A, B	held-out messages; save adapter

Ordinary LoRA still needs a dataset and a training run for each task.

PROGRAM-AS-WEIGHTS

Automatic adaptation per task

Can a model build the adapter for us?

Ordinary LoRA repeats training for every task

manual adaptation

collect examples

train LoRA

save adapter

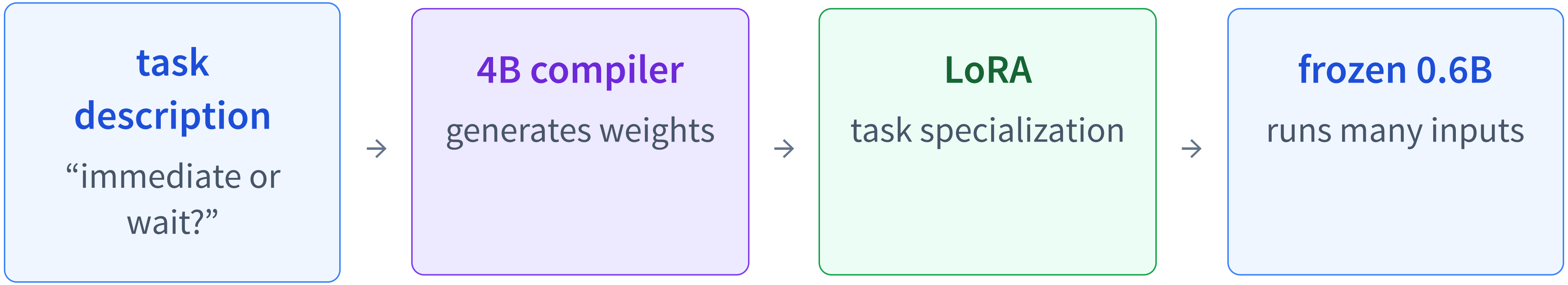
new possibility

describe the task

large model generates LoRA

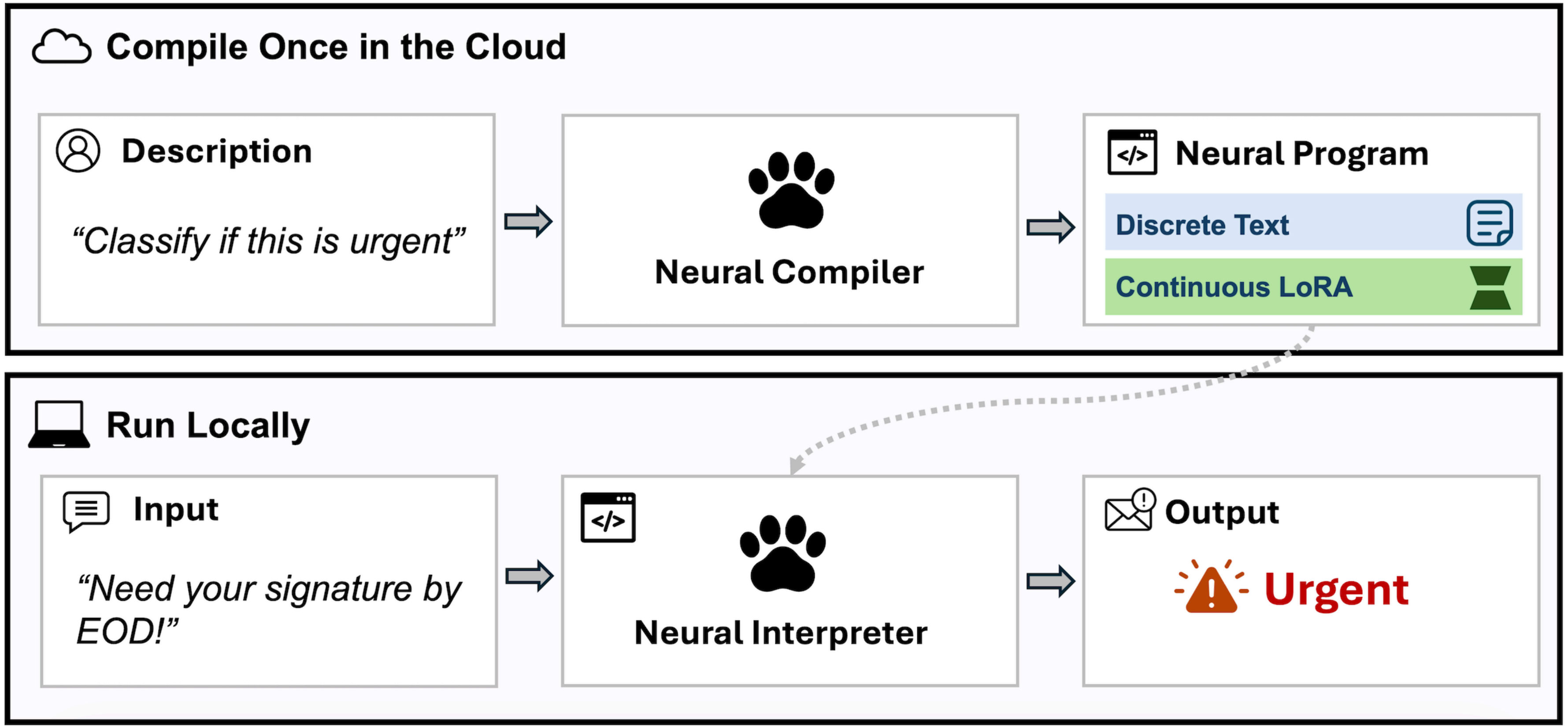
Use a large model once to build a reusable tool for a small model.

Program-as-Weights compiles a task into an adapter



The expensive model builds the function; the small interpreter executes it repeatedly.

Program-as-Weights: compile once, run repeatedly



The compiler builds a reusable neural program once; the interpreter then runs it on many inputs.

What trains the compiler?

Many tasks, each written as triples (s, x, y).

task	Classify immediate vs. wait
input	Need your signature today.
target	`immediate`

task	Classify sentiment
input	The update works beautifully.
target	`positive`

Across many tasks, the compiler learns how descriptions map to useful weights.

Train end to end through the frozen interpreter

task s
description

→

compiler
$s \mapsto \mathrm{LoRA}(s)$

→

frozen interpreter
$x \mapsto \hat{y}$

→

compare
$\hat{y}$ with target y

$$\mathcal{L} = -\log P(y \mid x, \mathrm{LoRA}(s))$$

Backpropagate through the frozen interpreter into the compiler.

Zhang et al., *Program-as-Weights*, 2026.

Compile an urgency adapter LIVE API

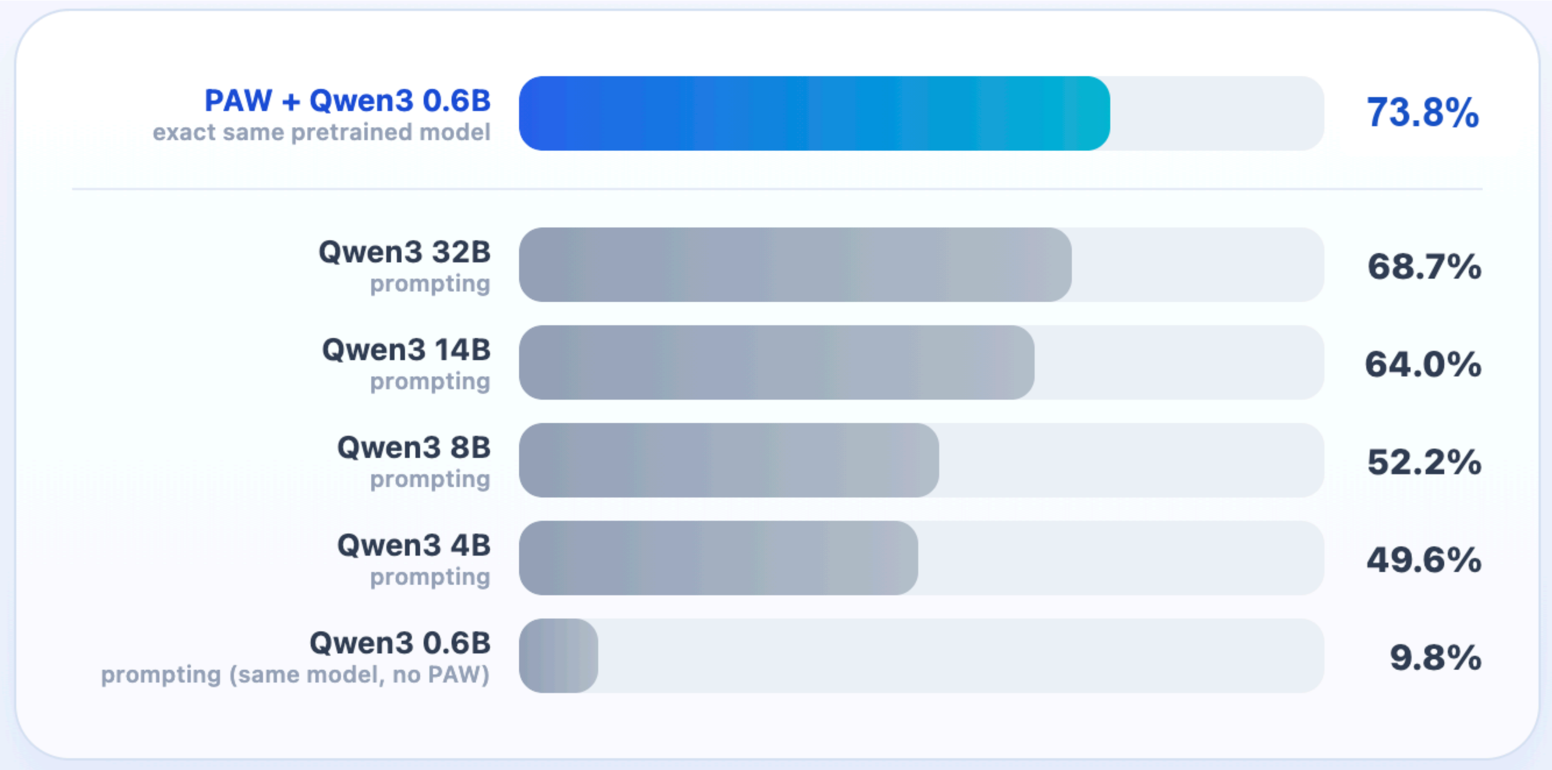
Describe `immediate` vs. `wait`, compile remotely once, then infer remotely on new messages.

describe immediate / wait → **compile remotely** → **infer remotely**

“Need your signature today” → **immediate**	“Newsletter for next month” → **wait**

The live demo uses the hosted PAW Standard compiler and infer endpoint.

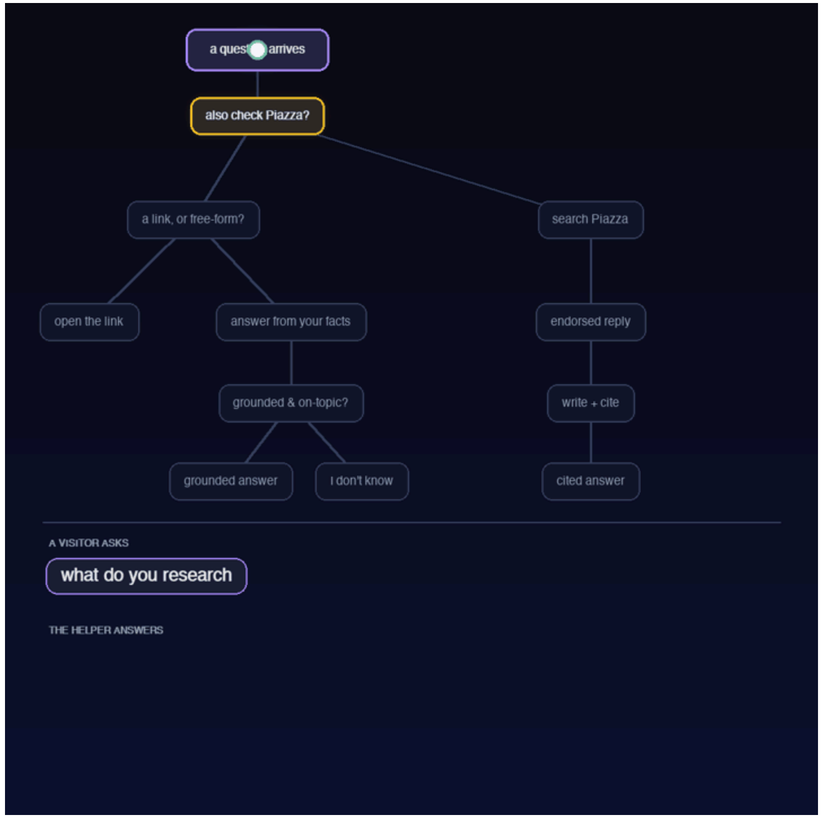
Compiler-generated LoRA can beat a much larger prompt



Verified FuzzyBench exact match: PAW reaches **73.8%** with a frozen 0.6B interpreter, above prompting a 32B model.

Final paper result: 73.78%, rounded to 73.8%.

The course helper is a neural decision tree



Ordinary software controls the branches; PAW neural programs implement semantic branch tests and features.

Compile once; call locally like a Python function

urgency.py

```python
import programasweights as paw

# Compile English into a local function.
fn = paw.compile_and_load(
    "Classify if a message needs immediate attention or can wait"
)

# Runs locally — no internet, no API call from here on.
fn("Thesis defense moved to 3pm, need your signature today")
# Returns "immediate"
```

compile once build the adapter **run repeatedly** use the frozen local interpreter

Why neural programs?

Count the verbs in: “**I can can the can.**”

can	**can**	**can**
modal verb	main verb	noun

answer: **2**

Learned programs are useful when behavior depends on meaning rather than brittle surface rules.

THE SHIFT

From AI as per-input problem solvers

↓

to AI as per-function tool builders

A large model compiles reusable specialization;
a small model runs it repeatedly.

Recap

- ✓ Prompting changes instructions; RAG retrieves evidence; tools execute actions.
- ✓ Fine-tuning changes model behavior through examples or preferences.
- ✓ LoRA freezes W_0 and learns the low-rank matrices A, B.
- ✓ Program-as-Weights compiles a task description into a reusable neural program.
- ✓ Diagnose what is missing, then choose the smallest useful adaptation.

L23: extend language models from text tokens to image tokens.